TP_modul_8

Nama : Helven Marcelyno Aldy Dwy Anggara

Nim : 108092500012

Kelas : IT-08-02

FILE CODE

1. queue_vaksinasi.h

main.cpp queue_vaksinasi.h

```cpp
#ifndef QUEUE_VAKSINASI_H
#define QUEUE_VAKSINASI_H

#include <iostream>
#include <string>

using namespace std;

#define nill nullptr

// Definisi Struktur Data
struct Infotype {
    string nama;
    int usia;
    string pekerjaan;
    bool prioritas;
    int nomor_antrean;
    int waktu_daftar;
    bool kondisi_darurat;
};

struct ElemQ {
    Infotype info;
    ElemQ *next;
};

struct Queue {
    ElemQ *head;
    ElemQ *tail;
};

// Makro untuk mempermudah akses
#define info(P) (P)->info
#define next(P) (P)->next
#define head(Q) (Q).head
#define tail(Q) (Q).tail

// ==========================================
// FUNGSI & PROSEDUR PRIMITIF
// ==========================================

void createQueue(Queue &Q) {
    head(Q) = nill;
    tail(Q) = nill;
}

bool isEmpty(Queue Q) {
    return head(Q) == nill;
}

ElemQ* createElemQueue(string nama, int usia, string pekerjaan, int nomor_antrean) {
    ElemQ *P = new ElemQ;
    info(P).nama = nama;
    info(P).usia = usia;
    info(P).pekerjaan = pekerjaan;
    info(P).prioritas = (usia >= 60 || pekerjaan == "tenaga kesehatan");
    info(P).nomor_antrean = nomor_antrean;
    info(P).kondisi_darurat = false;
    info(P).waktu_daftar = 0; // Default awal
    next(P) = nill;
    return P;
}
```

```cpp

void enqueue(Queue &Q, ElemQ *P) {
    if (isEmpty(Q)) {
        head(Q) = P;
        tail(Q) = P;
    } else if (info(P).prioritas) {
        // Jika elemen baru adalah prioritas, masukkan setelah prioritas yang sudah ada
        if (!info(head(Q)).prioritas) {
            next(P) = head(Q);
            head(Q) = P;
        } else {
            ElemQ *temp = head(Q);
            while (next(temp) != nill && info(next(temp)).prioritas) {
                temp = next(temp);
            }
            next(P) = next(temp);
            next(temp) = P;
            if (next(P) == nill) {
                tail(Q) = P;
            }
        }
    } else {
        // Jika non-prioritas, ditaruh di paling belakang
        next(tail(Q)) = P;
        tail(Q) = P;
    }
}

void dequeue(Queue &Q, ElemQ *&P) {
    if (isEmpty(Q)) {
        P = nill;
        cout << "Semua warga telah terlayani." << endl;
    } else {
        P = head(Q);
        head(Q) = next(head(Q));
        if (head(Q) == nill) {
            tail(Q) = nill;
        }
        next(P) = nill;
    }
}

ElemQ* front(Queue Q) {
    return head(Q);
}

ElemQ* back(Queue Q) {
    return tail(Q);
}

int size(Queue Q) {
    int count = 0;
    ElemQ *temp = head(Q);
    while (temp != nill) {
        count++;
        temp = next(temp);
    }
    return count;
}
```

```cpp
}

// ========================================
// FUNGSI & PROSEDUR NON-PRIMITIF
// ========================================

void printInfo(Queue Q) {
    if (isEmpty(Q)) {
        cout << "Antrean kosong." << endl;
        return;
    }
    cout << "Daftar Antrean:" << endl;
    ElemQ *temp = head(Q);
    while (temp != nill) {
        cout << "Nama: " << info(temp).nama << endl;
        cout << "Usia: " << info(temp).usia << endl;
        cout << "Pekerjaan: " << info(temp).pekerjaan << endl;
        cout << "Prioritas: " << (info(temp).prioritas ? "Ya" : "Tidak") << endl;
        cout << "Nomor Antrean: " << info(temp).nomor_antrean << endl;
        temp = next(temp);
    }
}

void serveQueue(Queue &Q) {
    int kapasitas = 0;
    ElemQ *P = nill;

    while (!isEmpty(Q) && kapasitas < 100) {
        dequeue(Q, P);
        if (P != nill) {
            cout << "Melayani warga:" << endl;
            cout << "Nama\t\t: " << info(P).nama << endl;
            cout << "Usia\t\t: " << info(P).usia << endl;
            cout << "Pekerjaan\t: " << info(P).pekerjaan << endl;
            cout << "Prioritas\t: " << (info(P).prioritas ? "Ya" : "Tidak") << endl;
            cout << "Vaksinasi berhasil.\n" << endl;
            delete P;
            kapasitas++;
        }
    }

    if (kapasitas == 100 && !isEmpty(Q)) {
        cout << "Kapasitas telah penuh. Warga yang belum terlayani diminta kembali besok." << endl;
    }
}

void reassignQueue(Queue &Q) {
    Queue tempQ;
    createQueue(tempQ);
    ElemQ *P = nill;

    while (!isEmpty(Q)) {
        dequeue(Q, P);
        // Memastikan status prioritas dihitung kembali sesuai kategori usia/pekerjaan
        info(P).prioritas = (info(P).usia >= 60 || info(P).pekerjaan == "tenaga kesehatan");
        enqueue(tempQ, P);
```

```
    }
    Q = tempQ;
}

void checkWaitingTime(Queue &Q, int waktu_sekarang) {
    Queue tempQ;
    createQueue(tempQ);
    ElemQ *P = nill;

    while (!isEmpty(Q)) {
        dequeue(Q, P);
        int waktu_tunggu = waktu_sekarang - info(P).waktu_daftar;
        if (waktu_tunggu > 120) {
            info(P).prioritas = true;
        }
        enqueue(tempQ, P);
    }
    Q = tempQ;
}

ElemQ* findAndRemove(Queue &Q, int nomor_antrean) {
    Queue tempQ;
    createQueue(tempQ);
    ElemQ *P = nill;
    ElemQ *found = nill;

    while (!isEmpty(Q)) {
        dequeue(Q, P);
        if (info(P).nomor_antrean == nomor_antrean && found == nill) {
            found = P; // Pisahkan elemen yang dicari, tidak dikembalikan ke tempQ
        } else {
            enqueue(tempQ, P);
        }
    }
    Q = tempQ;

    if (found == nill) {
        cout << "Warga dengan nomor antrean " << nomor_antrean << " tidak ditemukan dalam antrean." << endl;
    }
    return found;
}

void emergencyHandle(Queue &Q, int nomor_antrean) {
    ElemQ *P = findAndRemove(Q, nomor_antrean);
    if (P != nill) {
        info(P).kondisi_darurat = true;
        info(P).prioritas = true;
        // Memaksa elemen diletakkan di posisi paling depan queue
        if (isEmpty(Q)) {
            head(Q) = P;
            tail(Q) = P;
            next(P) = nill;
        } else {
            next(P) = head(Q);
            head(Q) = P;
        }
    } else {
        // Output mengikuti format modul saat elemen tak ditemukan
    }
}

void updatePriority(Queue &Q) {
    Queue tempQ, emergQ;
    createQueue(tempQ);
    createQueue(emergQ);
    ElemQ *P = nill;

    // Pisahkan elemen yang darurat dengan yang tidak
    while (!isEmpty(Q)) {
        dequeue(Q, P);
        if (info(P).kondisi_darurat) {
            enqueue(emergQ, P);
        } else {
            enqueue(tempQ, P);
        }
    }

    // Masukkan kembali mulai dari kondisi darurat, lalu sisanya
    // Fungsi primitive enqueue akan otomatis mengurutkan prioritas sisanya
    while (!isEmpty(emergQ)) {
        dequeue(emergQ, P);
        enqueue(Q, P);
    }
    while (!isEmpty(tempQ)) {
        dequeue(tempQ, P);
        enqueue(Q, P);
    }
}

#endif
```

main.cpp

main.cpp queue_vaksinasi.h

```cpp
#include <iostream>
#include "queue_vaksinasi.h"

using namespace std;

int main() {
    // Membuat antrean
    Queue Q;
    createQueue(Q);

    // Menambahkan beberapa elemen ke dalam queue
    ElemQ* P1 = createElemQueue("John Doe", 65, "lansia", 1);
    ElemQ* P2 = createElemQueue("Alice", 30, "tenaga kesehatan", 2);
    ElemQ* P3 = createElemQueue("Bob", 25, "pekerja", 3);
    ElemQ* P4 = createElemQueue("Charlie", 70, "pensiunan", 4);
    ElemQ* P5 = createElemQueue("David", 28, "pekerja", 5);

    enqueue(Q, P1);
    enqueue(Q, P2);
    enqueue(Q, P3);
    enqueue(Q, P4);
    enqueue(Q, P5);

    // Menampilkan isi queue
    cout << "Isi antrean awal:" << endl;
    printInfo(Q);

    // Melayani antrean
    cout << "\nMelakukan pelayanan pada antrean:" << endl;
    serveQueue(Q);

    // Memeriksa antrean setelah pelayanan
    cout << "Isi antrean setelah pelayanan:" << endl;
    printInfo(Q);

    // Menambahkan elemen baru untuk simulasi pengaturan ulang prioritas
    ElemQ* P6 = createElemQueue("Edward", 22, "pekerja", 6);
    enqueue(Q, P6);

    // Simulasi reassignQueue untuk mengatur ulang antrean ke prioritas
    cout << "\nMengatur ulang antrean berdasarkan prioritas:" << endl;
    reassignQueue(Q);
    printInfo(Q);

    // Simulasi kondisi warga yang menunggu lebih dari 2 jam
    cout << "\nMemeriksa waktu tunggu dan mengubah prioritas jika lebih dari 2 jam:" << endl;
    checkWaitingTime(Q, 130);
    printInfo(Q);

    // Menangani kondisi darurat untuk seorang warga
    cout << "\nMenangani kondisi darurat untuk warga dengan nomor antrean 5:" << endl;
    emergencyHandle(Q, 5);
    printInfo(Q);

    // Update prioritas warga setiap jam
    cout << "\nMengupdate prioritas antrean setiap jam:" << endl;
    updatePriority(Q);
    printInfo(Q);

    // Mencari dan menghapus warga dengan nomor antrean tertentu
    cout << "\nMenghapus warga dengan nomor antrean 3:" << endl;
    ElemQ* removedElem = findAndRemove(Q, 3);
    if (removedElem) {
        cout << "Warga yang dihapus: " << info(removedElem).nama << endl;
        delete removedElem; // Menghindari kebocoran memori setelah diambil
    }
    printInfo(Q);

    // Mengecek ukuran queue
    cout << "\nUkuran antrean saat ini: " << size(Q) << endl;

    return 0;
}
```

BUILD & RUN

```
Isi antrean awal:
Daftar Antrean:
Nama: John Doe
Usia: 65
Pekerjaan: lansia
Prioritas: Ya
Nomor Antrean: 1
Nama: Alice
Usia: 30
Pekerjaan: tenaga kesehatan
Prioritas: Ya
Nomor Antrean: 2
Nama: Charlie
Usia: 70
Pekerjaan: pensiunan
Prioritas: Ya
Nomor Antrean: 4
Nama: Bob
Usia: 25
Pekerjaan: pekerja
Prioritas: Tidak
Nomor Antrean: 3
Nama: David
Usia: 28
Pekerjaan: pekerja
Prioritas: Tidak
Nomor Antrean: 5

Melakukan pelayanan pada antrean:
Melayani warga:
Nama            : John Doe
Usia            : 65
Pekerjaan       : lansia
Prioritas       : Ya
Vaksinasi berhasil.

Melayani warga:
Nama            : Alice
Usia            : 30
Pekerjaan       : tenaga kesehatan
Prioritas       : Ya
Vaksinasi berhasil.

Melayani warga:
Nama            : Charlie
Usia            : 70
Pekerjaan       : pensiunan
Prioritas       : Ya
Vaksinasi berhasil.

Melayani warga:
Nama            : Bob
Usia            : 25
Pekerjaan       : pekerja
Prioritas       : Tidak
Vaksinasi berhasil.

Melayani warga:
Nama            : David
Usia            : 28
Pekerjaan       : pekerja
Prioritas       : Tidak
Vaksinasi berhasil.

Isi antrean setelah pelayanan:
Antrean kosong.
```

```
Mengatur ulang antrean berdasarkan prioritas:
Daftar Antrean:
Nama: Edward
Usia: 22
Pekerjaan: pekerja
Prioritas: Tidak
Nomor Antrean: 6

Memeriksa waktu tunggu dan mengubah prioritas jika lebih dari 2 jam:
Daftar Antrean:
Nama: Edward
Usia: 22
Pekerjaan: pekerja
Prioritas: Ya
Nomor Antrean: 6

Menangani kondisi darurat untuk warga dengan nomor antrean 5:
Warga dengan nomor antrean 5 tidak ditemukan dalam antrean.
Daftar Antrean:
Nama: Edward
Usia: 22
Pekerjaan: pekerja
Prioritas: Ya
Nomor Antrean: 6

Mengupdate prioritas antrean setiap jam:
Daftar Antrean:
Nama: Edward
Usia: 22
Pekerjaan: pekerja
Prioritas: Ya
Nomor Antrean: 6

Menghapus warga dengan nomor antrean 3:
Warga dengan nomor antrean 3 tidak ditemukan dalam antrean.
Daftar Antrean:
Nama: Edward
Usia: 22
Pekerjaan: pekerja
Prioritas: Ya
Nomor Antrean: 6

Ukuran antrean saat ini: 1

Process returned 0 (0x0)   execution time : 0.447 s
Press any key to continue.
```